

Algorithmes et Pensée Computationnelle

Programmation de base

Le but de cette séance est d'aborder des notions de base en programmation. Au terme de cette séance, vous serez capable de :

- Comprendre les différentes représentations de nombres.
- Définir des variables et comprendre les types de variables existants.
- Utiliser des notions d'algèbre booléenne.
- Ecrire des branchements conditionnels.

Les langages qui seront utilisés pour cette séance sont Java et Python. Assurez-vous d'avoir bien installé IntelliJ. Si vous rencontrez des difficultés, n'hésitez pas à vous référer au guide suivant : [tutoriel d'installation des outils \(MacOS\)](#) et [prise en main de l'environnement de travail](#) ou [tutoriel d'installation des outils \(Windows\)](#).

1 Input / Output

Question 1: (🕒 5 minutes) Output (Java ou Python)

Créez une variable *nom* (str) contenant votre nom, et une autre *prenom* (str) contenant votre prénom puis affichez : "Bonjour, *prenom nom*".

💡 Conseil

Utilisez la fonction `print()` de Python et `System.out.println()` de Java.

Question 2: (🕒 5 minutes) Input (Java ou Python)

En vous référant à l'exercice précédent (**Output (Java ou Python)**), demandez à l'utilisateur d'entrer son nom et son prénom via la fonction `input()` au lieu d'initialiser vous-même les variables.

💡 Conseil

Utilisez la fonction `input()` en Python, la classe `Scanner()` en Java (n'oubliez pas d'ajouter `import java.util.Scanner;` tout au début de votre code).

2 Représentation de nombres entiers

Question 3: (🕒 5 minutes) Entiers non signés

Sur 8 bits, convertir $113_{(10)}$ en base binaire.

💡 Conseil

Faire un tableau comme présenté dans la diapositive 13 du cours (programmation de base). Essayer de décomposer le nombre en une somme de puissances de 2.

Question 4: (🕒 5 minutes) Entiers signés négatifs

En utilisant le résultat de la question précédente, convertir sur 8 bits $-113_{(10)}$ en base binaire. Utiliser la représentation **signée** à la diapositive 13 du cours.

 Conseil

- Il faut prendre la représentation sur 7 bits d'un nombre entier non signé.
- On rajoute un 8ème bit qui sera le signe.
- Le premier bit à gauche vaut 0 pour un nombre positif et 1 pour un nombre négatif.

Question 5: (🕒 5 minutes) **Complément à 1**

Écrire le complément à 1 de $-113_{(10)}$ sur 8 bits.

Quelle est la différence entre cette méthode et la précédente ?

 Conseil

- L'opposé de 0 en binaire est 1 et inversement.
- Pour étudier la différence, il faut regarder les différentes manières d'exprimer -0 en binaire (se référer à la diapositive 14 du cours).

Question 6: (🕒 5 minutes) **Complément à 2**

Quelle est la représentation du complément à 2 (two complement) de $-113_{(10)}$ sur 8 bits et quelle est l'utilité de cette représentation ?

 Conseil

Exemple du cours avec $87_{(10)}$

$$87_{(10)} = 01010111_{(2)}$$

a	0	1	0	1	0	1	1	1
b	1	0	1	0	1	0	0	0
c	1	0	1	0	1	0	0	1

a : convertir le nombre en binaire

b : inverser tous les bits (en utilisant **not**)

c : rajouter 1 à b pour obtenir le complément à 2

On obtient donc $-87_{(10)} = 10101001_{(2)}$

Question 7: (🕒 10 minutes) **Floating point**

Voici la représentation en binaire d'un nombre à virgule flottante :

signe	exposant	mantisse
0	10110101	010000010000000000000001

Que vaut cette représentation en base 10 ? Utiliser la représentation des floating points (avec un biais de 127). Arrondir les résultats intermédiaires et la valeur finale au 3ème chiffre significatif après la virgule.

Conseil

- Se référer à la diapositive 21 du cours pour plus de détails.
- Poser chaque calcul, et ensuite tout fusionner avec la formule.
- L'exposant et la mantisse sont des entiers positifs, donc non signés.
- Pour vérifier votre résultat, vous pouvez utiliser l'outil suivant : <https://www.h-schmidt.net/FloatConverter/IEEE754.html>

3 Typage

Le but de cette partie des travaux pratiques est de comprendre les notions de typage statique et dynamique, ainsi que leur impact sur l'exécution et la gestion des erreurs.

Question 8: (🕒 5 minutes) Les types de variables

Quelles sont les principaux types de variables (donnez en 3), comment les déclareriez-vous en Python et en Java ?

Conseil

Se référer aux diapositives du cours ou à la documentation officielle des langages de programmation.

Question 9: (🕒 10 minutes) Typage statique et dynamique

Parmi ces différents programmes, lesquels pourront être exécutés sans problème et lesquels lèveront des erreurs ? Dans le cas où ils génèreraient des erreurs, expliquez la raison de ces erreurs.

Java :

```
1 // Programme 1
2 int variable_1 = 0;
3 int variable_2 = 3;
4 variable_2 = variable_1;
5
6 // Programme 2
7 var variable_1 = 0;
8 var variable_2 = "Hello";
9 variable_2 = variable_1;
10
11 // Programme 3
12 int variable_1 = 0;
13 String variable_2 = "Hello";
14 variable_2 = variable_1;
15
16 // Programme 4
17 var variable_1 = 0;
18 variable_1 = 3.14 ;
19
20 // Programme 5
21 var variable_1 = 0;
22 String variable_2 = "Hello";
23 String variable_3 = "World";
24 variable_2 = variable_3;
```

```

25
26 // Programme 6
27 int variable_1 = 0;
28 variable_1 = 3.14 ;

```

Python :

```

1 # Programme 7
2 variable_1 = 0
3 variable_2 = "Hello"
4 variable_2 = variable_1
5
6 # Programme 8
7 variable_1 = 0
8 variable_1 = 3.14

```

💡 Conseil

En Java, le type est statique, ce qui signifie qu'à partir du moment où vous créez une variable, un type va lui être attribué (par vous ou par le code en lui-même par déduction). Ce type ne pourra pas être changé, et si vous tentez de le faire (en lui attribuant une valeur d'un autre type par exemple), une erreur sera levée.

En Python, le typage est dynamique, il est donc tout à fait possible de changer le type d'une variable sans déclencher d'erreur.

⚠ Erreurs fréquentes

Au cas où vous exécuteriez les programmes 2, 4 et 5 et rencontrez l'erreur suivante : `Exception in thread "main" java.lang.UnsupportedClassVersionError: ... has been compiled by a more recent version of the Java Runtime (class file version 54.0), this version of the Java Runtime only recognizes class file versions up to 52.0`, suivez les étapes ci-dessous :

1. Mettez à jour votre JDK en téléchargeant la version adéquate via le lien suivant : <https://www.oracle.com/java/technologies/downloads/>.
2. Une fois votre JDK installé, redémarrez IntelliJ
3. Dans la barre de menus, cliquez sur **⚙ > Edit...**
4. Dans la fenêtre qui apparaîtra (capture ci-dessous), sélectionnez la version du JRE la plus récente.

4 Opérateurs et conditions Booléennes

Le principe d'une valeur booléenne est qu'elle ne puisse contenir que 2 valeurs possibles, soit `True`, soit `False`. Il est possible de les définir en leur associant une de ces valeurs d'emblée ou de les obtenir en effectuant une opération booléenne comme par exemple une comparaison. Pour ce faire, il faut utiliser des opérateurs booléens. Voici les plus utilisés : `=` (est égal), `!=` (n'est pas égal), `<` (est strictement plus petit), `≤` (est plus petit ou égal), `>` (est strictement plus grand), `≥` (est plus grand ou égal). Si la condition est satisfaite, on obtiendra `True`, si elle ne l'est pas, on obtiendra `False`. L'utilisation de l'opérateur `not` inversera le résultat.

Question 10: (🕒 7 minutes)

1. Démontrez l'équivalence suivante :

$$(\neg p \vee q) \wedge (p \vee \neg q) \Leftrightarrow (p \wedge q) \vee (\neg p \wedge \neg q)$$

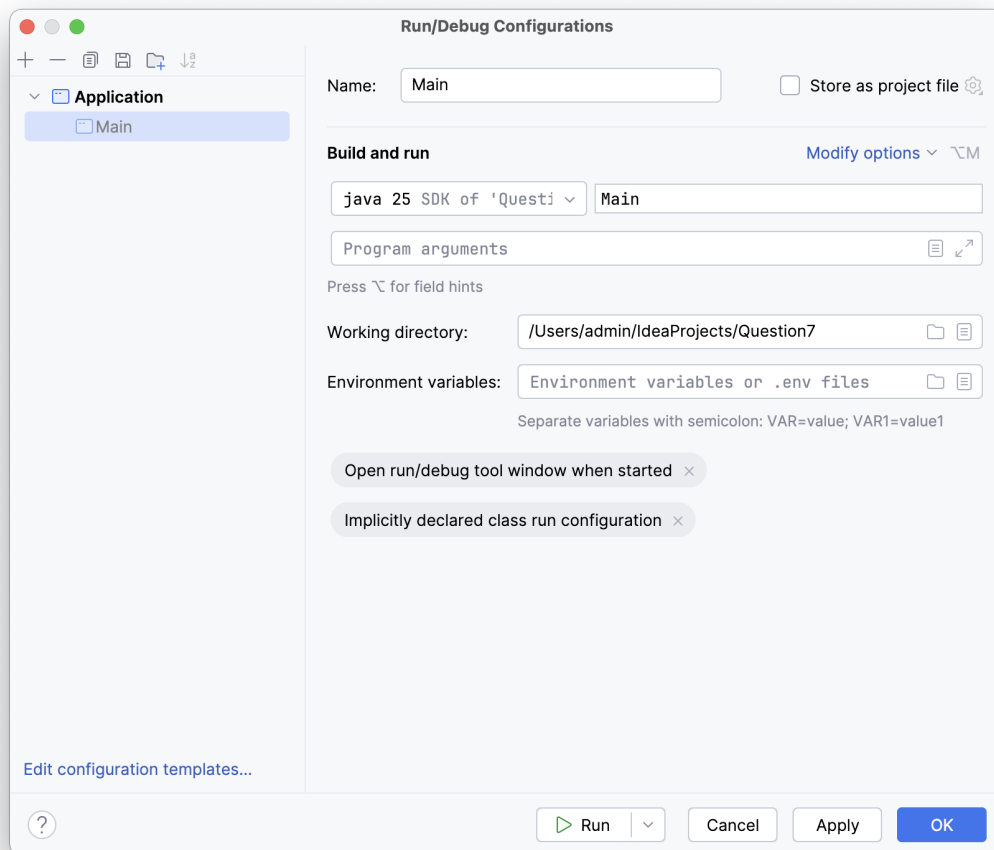


FIGURE 1 – Fenêtre de configuration de l’environnement de développement Java.

Conseil

Utilisez la table des règles présentées à la diapositive 29 du cours pour déduire cette équivalence.

2. Re-faire cet exercice en utilisant les tables de vérité.

Dans les exercices suivants, vous devrez anticiper la valeur que la console va vous donner (résultat du print).

Question 11: (🕒 5 minutes) Qu’affiche le programme suivant ?

Conseil

Utilisez les tables de vérité présentées à la diapositive 28 du cours.

```

1 a = 3
2 b = 2
3 c = 6
4 d = c >= b
5 print(a==b or d)
6 print(a==b and d)
7 print(a==b or a==c or False or d)
8 print(a<c and not (4*b==8 and not d))

```

Question 12: (🕒 5 minutes)

1. Qu'affiche le programme suivant ?

```
1 a = True
2 b = False
3 c = True
4 if a or (b and not c):
5     print("output 1")
6 if b != c :
7     print("output 2")
8 if a or (not b != (c and a)):
9     print("output 3")
10 else:
11     print("output 4")
```

2. Qu'affiche le programme si l'on change la ligne 6 à `elif b != c` ?

5 Match (Python) et Switch (Java)

L'instruction `match` en Python et `switch` en Java a pour objectif de simplifier la logique conditionnelle en sélectionnant un bloc de code à exécuter parmi plusieurs options. Cette structure compare la valeur d'une variable ou d'une expression à une série de cas (case) prédéfinis. Lorsqu'une correspondance est trouvée, le code associé à ce cas est exécuté, offrant ainsi une alternative plus lisible et organisée qu'une longue chaîne de `if-elif-else`.

Consultez [cette mémoire disponible sur Moodle](#) pour comprendre le fonctionnement d'un `switch` en Java.

Question 13: (🕒 5 minutes) Écrire le code Java qui demande à l'utilisateur d'entrer l'étape d'exécution d'une instruction et affiche les étapes qui reste à exécuter (y compris l'étape courante). Si l'utilisateur entre une étape invalide le code affiche L'étape n'est pas valide.

💡 Conseil

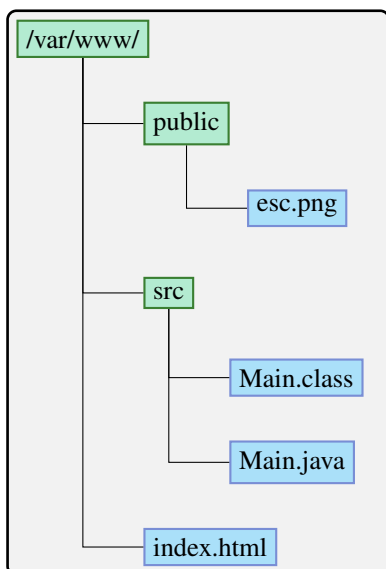
Rappelez-vous de l'architecture de Von Neumann disponible à la diapositive 17 du [premier cours](#). Pour lire la saisie texte de l'utilisateur, utilisez `nextLine()` telle qu'elle est définie dans [la documentation Java en ligne](#).

Exemple

Si l'utilisateur entre `execute`, le programme va afficher :

L'étape `execute` s'exécute ou va être exécutée.
L'étape `store` s'exécute ou va être exécutée.

Question 14: (🕒 5 minutes)



Soit l'arborescence à gauche d'un système de fichiers (file-system). Écrivez un programme Python (utilisant `match`) qui affiche **seulement les noms des fichiers** dans un seul dossier spécifié à la ligne de commande. Votre programme doit seulement gérer les arguments suivants :

- `/var/www/./public/..`
- `/var/www/src/..`
- `/var/www/src/`
- `/var/www/public/`

Pour les autres cas, il faut afficher La saisie n'était pas reconnue.

💡 Conseil

Pour spécifier un argument à la ligne de commande en Python [consultez cette documentation en ligne](#). Pour le `match` en Python, consultez [la diapositive 37 du cours](#).

Re-faire cet exercice en utilisant `input` pour lire la saisie de l'utilisateur.

💡 Conseil

La documentation de `input` est [disponible sur ce lien](#).

6 Type casting

Question 15: (🕒 5 minutes) Conversion des variables (Type casting) (Java ou Python)

Qu'affichent les programmes suivants ?

Python :

```

1 nombre_entier = 3
2 nombre_decimal = float(nombre_entier)
3 print(nombre_entier)
4 print(nombre_decimal)

```

Java :

```

1 float nombre_decimal = 3.14f;
2 int nombre_entier = (int) nombre_decimal;
3 System.out.println(nombre_entier);
4 System.out.println(nombre_decimal);

```

💡 Conseil

Attention, ces fonctions ne changent pas le type des variables, elles ne font que les convertir.

7 Chaîne de caractères

Question 16: (🕒 10 minutes)

Soit la chaîne de caractères : UNIL\nSchool of Criminal Sciences.

myString=“	U	N	I	L	\n	S	c	h	o	...	”
indices	0	1	2	3	4	5	6	7	8	9 à 31	

⚠ Attention

Ici, le caractère `\` est seulement pour mettre en évidence le caractère espace.

1. Affichez cette chaîne en Python et en Java. Pourquoi est-elle séparée par une nouvelle ligne ?
2. Essayez d’afficher uniquement la chaîne School of Crimi en sélectionnant une sous-chaîne de la chaîne initiale.

💡 Conseil

- Pour créer une sous-chaîne en Java, utilisez la méthode `myString.substring(int beginIndex, int endIndex)` où `beginIndex` est l’indice du premier caractère dans la chaîne `myString` que l’on veut inclure dans la sous-chaîne et `endIndex` est l’indice du caractère suivant le dernier à inclure (ce caractère n’est pas inclus dans la sous-chaîne).
- Pour créer une sous-chaîne en Python, utilisez l’opérateur slice `myString[startIndex, endIndex]` et pareil pour l’interprétation du `beginIndex`, `endIndex`, et `myString`.

3. Accédez au troisième caractère de la chaîne en Python et en Java.

💡 Conseil

- Pour obtenir le *i*ème caractère en Java, on écrit `myString.charAt(i)`.
- Pour obtenir le *i*ème caractère en Python, on écrit `myString[i]`.